



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

第九届全国大学生计算机系统能力培养大赛

“龙芯杯”竞赛总结报告（个人赛）

指令集： LoongArch

姓 名： 贾又维

2025 年 09 月 06 日

目 录

一、概述.....	1
1.1 初赛/决赛指标	1
1.2 设计概述.....	1
二、系统详细设计.....	2
2.1 系统整体架构设计.....	2
2.2 流水线结构设计.....	2
2.3 分支预测器设计.....	4
2.4 冒险和发射策略设计.....	4
2.5 指令缓存设计.....	5
2.6 指令队列设计.....	5
2.7 总线模块设计.....	6
三、关键技术及特色/亮点	7
四、调试及优化.....	7
4.1 测试方法.....	7
4.2 调试和优化过程.....	7
五、复赛编程题设计.....	8
六、关键问题及解决方案.....	9
七、总结与展望.....	9
7.1 竞赛总结.....	9
7.2 未来展望.....	9
7.3 备赛经验及建议.....	9

一、概述

1.1 初赛/决赛指标

初赛性能测试：

STREAM	MATRIX	CRYPTONIGHT	总用时
0.024s	0.055s	0.118s	0.197s

CPU 频率：262M Hz

决赛性能测试：

STREAM	MATRIX	CRYPTONIGHT	总用时
0.021s	0.054s	0.113s	0.188s

CPU 频率：265M Hz

决赛答题时长：0.013s

1.2 设计概述

学习路线：在寒假跟着雷思磊的《自己动手写 CPU》完成了前九章的内容，大致了解了五级流水线的基本结构和部分数据冒险和控制冒险。春季学期参与竞赛指导课程进一步学习了五级流水线还有分支预测器和 cache 相关内容。在完成了一个完整的带有分支预测器和指令数据 cache 的五级流水线后，通过姚永斌的《超标量处理器设计》学习了在设计超标量 cpu 的一些设计方向，最后实现了一个重命名到 ROB 的乱序双发射 cpu（因为没有实现提前唤醒机制和 checkpoint 分支预测错误恢复机制导致表现不如五级流水线）。最后在测试平台开放后发现我写的乱序 cpu 不仅 ipc 低，主频也提不上去，就在将五级流水线超频超到 210M 后重新写了一个顺序双发射 cpu。

系统总体架构：最终实现的是一个九级顺序双发射 CPU，分为四级取指，译码，发射，执行，访存，写回九个阶段。实现了 LoongArch32 除异常浮点指令外的全部指令，并且添加了指令 cache 和动态分支预测器，没有数据 cache。

二、系统详细设计

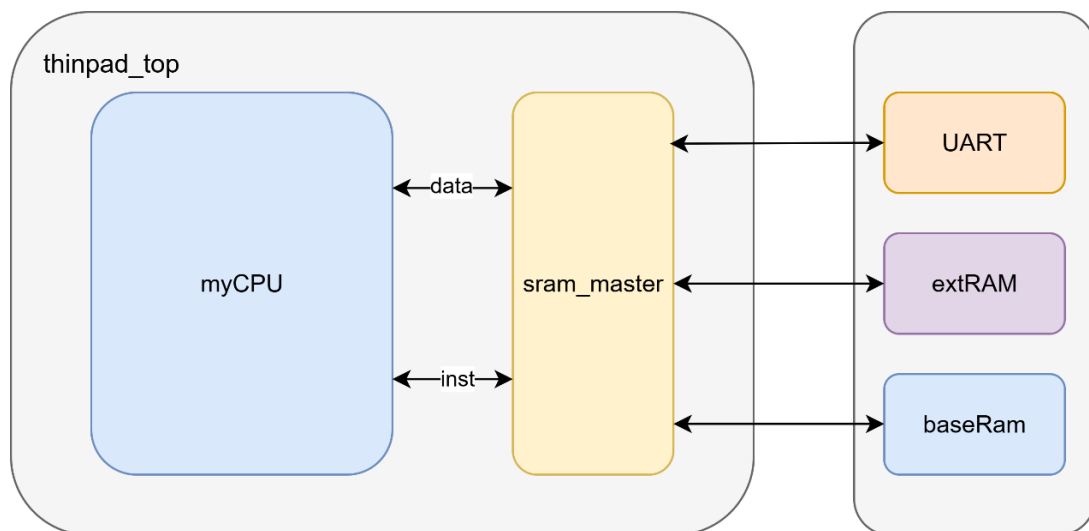
2.1 系统整体架构设计

本设计的顶层将整个系统划分为三大核心部分：CPU 核（myCPU）、总线仲裁与存储控制器（sram_master）以及 串口控制器。

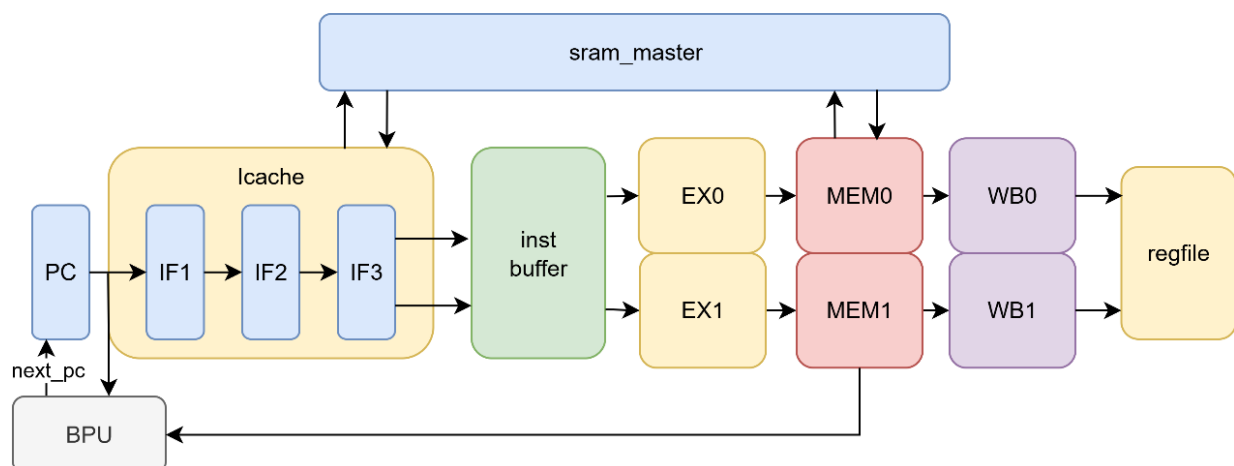
1. CPU 核心 (mycpu): 采用九级流水、双发射超标量结构。

2. 总线仲裁与存储控制器 (sram_master): 是连接 CPU 核与物理存储、外设的桥梁。

3. 串口控制器: 使用的大赛提供的 async_receiver 和 async_transmitter。



2.2 流水线结构设计



流水线被划分为以下九个阶段：

1. 前端 - 4 级取指 (IF1-IF4):

IF0: PC 寄存器生成取指地址，同时分支预测单元对该地址进行动态预测生成下一个 PC。同时本周期的 PC 值立刻用于访问指令缓存的存储阵列，启动缓存行读取。

IF1: BRAM 在本周期返回缓存行数据和存储的 Tag。流水线此级负责将读出的 Tag 与当前指令的 Tag 进行比较，判断是否命中。

IF2: 根据上一级的命中信号进行决策。若为缓存缺失，则启动缺失处理状态机，暂停前端流水线并向总线发出主存读取请求；若命中，则将数据和控制信号传递给下一级。

IF3: 本级负责对读出的 64 位缓存行数据进行对齐。根据 PC 地址和分支预测结果，提取出最多两条有效指令，连同其 PC 和预测信息，发送至指令缓冲队列。

2. 译码和发射 - 2 级:

指令缓冲与译码: 从 ICache 取出的原始指令进入一个 4 项深的指令缓冲队列。在该阶段，指令被并行译码，生成后续阶段所需的所有控制信号，并等待发射。该缓冲队列有效解耦了前端取指和后端执行的速度差异。

发射: 发射逻辑检查指令缓冲队列头部的两条已译码指令，判断是否存在数据冒险或结构冒险。根据判断结果，决定在该周期向后端发射零条、一条或两条指令。

3. 后端 - 3 级执行 (EX, MEM, WB):

执行: 发射出的指令进入两个并行的执行通路。ALU 执行算术逻辑运算，同时计算出分支指令的实际跳转结果。

访存: 处理访存指令，通过总线接口与总线控制器交互。同时，将 EX 阶段的分支实际结果与 IF1 阶段的预测结果进行比较，若预测错误，则产生冲刷信号，清空前端和译码发射阶段，并用正确的目标地址更新 PC 和 BPU。

写回: 将 ALU 的计算结果或从内存读取的数据写回寄存器堆。

2.3 分支预测器设计

1. 预测路径（组合逻辑）：

为确保单周期预测，此路径为纯组合逻辑。因为一次最多需要取两条指令，所以同时需要对两条指令进行预测，所以每次预测都需要并行的访问两个 BTB（分别对应 PC [2] 为 0 和 1）来查询预测信息。在 IF1 阶段，BPU 利用 PC 的 [22:5] 位作为 Tag、[4:3] 位作为索引，并行访问 BTB 的两路存储。当某一路命中（Tag 匹配且 valid）且其 2 位饱和计数器预测为跳转时，该路即产生有效的跳转预测。最终，仲裁逻辑综合两路结果，输出最终的预测信号、目标地址和跳转指令在包内的偏移。

2. 更新路径（两级流水）：

当一条分支指令在 MEM 阶段提交时，其真实结果被送入 BPU 的更新接口。（分支指令计算出结果再和预测结果进行对比形成了一条关键路径，所以我将预测结果的验证移到了 MEM 阶段，这个会增加分支预测错误的惩罚，但对于个人赛的三个性能测试没有影响）

S1 级：根据提交的 PC 值计算地址，并从 BPU 存储阵列中读出旧的条目数据。

S2 级：根据实际情况增加替换条目或者更新 2 位饱和计数器

2.4 冒险和发射策略设计

指令发射顺序：在指令发射阶段，处理器每个周期最多尝试发射两条指令，并严格遵循程序顺序。流水线 pip0 中的指令在程序顺序上总是先于 pip1 中的指令。

数据冒险处理：

- **数据前递：**为了解决 RAW 冒险，发射阶段待发射指令的源寄存器会与 EX、MEM、WB 阶段所有指令（共 6 条）的目的寄存器进行比较。如果匹配，相应的结果会通过前递路径直接转发给执行单元，避免不必要的暂停。

- **Load-Use 冒险：**当指令需要使用前一条 Load 指令的结果时，由于数据在 MEM 阶段才可用，流水线必须暂停一个周期。

- **发射包内部依赖：**如果在同一个发射包内，pip1 指令依赖于 pip0 指令的

结果，则该周期仅发射 pip0 指令。被阻塞的 pip1 指令将在下一个周期成为新的 pip0 候选指令，并与后续指令重新尝试双发射。

结构冒险处理:

- **访存单元:** 两条流水线共享同一个访存模块，因此不能在同一周期内发射两条访存指令。
- **分支单元:** 分支预测器的更新逻辑为单端口，因此不支持在同一周期内发射两条分支指令。

2.5 指令缓存设计

1. 缓存组织结构

- **映射方式:** 直接映射。
- **存储实现:** 缓存的数据存储体由 FPGA 内部的块存储器(BRAM)实现。
- **容量与尺寸:** 总数据容量 256 字节，在逻辑上划分为 32 个缓存行，每行大小为 64 位。

2. Cache 缺失处理

当发生 Cache Miss 时，一个状态机将接管控制，执行以下步骤：

- **暂停流水线:** 暂停 PC 的更新。
- **请求主存:** 向主存发送读请求，地址为当前缺失的地址对齐后的块地址。
- **等待数据:** 等待主存返回数据。
- **数据填充:** 收到数据后，将新的数据块以及新的 Tag 写入到 BRAM 中由索引指定的 Cache 行，并设置有效位。
- **恢复执行:** 将从主存返回的数据直接提供给 CPU，同时流水线恢复正常运行。

2.6 指令队列设计

1. 核心功能

该模块的主要功能是作为一个深度为 4 的 FIFO，用于缓存从前端取回的、尚未译码的原始指令包。它避免取指部件的停顿直接影响到后续流水线，从而提高处理器的整体效率。

2. 主要特性

（1）双输入双输出：

- **输入端：**每个周期最多可以接收两个有效的指令。
- **输出端：**模块前端维持两个可供消耗的指令槽位，后续阶段每个周期可消耗零个、一个或两个指令包（由发射阶段传回来的控制信号控制）。

（2）输出端的流水线级：

• **组合逻辑阶段：**在一个时钟周期内，模块的组合逻辑完成“读取-译码-选择”的完整操作。它从 FIFO 头部读取原始指令，通过译码模块进行译码，并根据 pop 信号和当前 front 寄存器的状态，计算出下一周期应该在 front1 和 front2 槽位上显示的、已译码的指令。

• **寄存器锁存阶段：**在时钟上升沿，上述组合逻辑的计算结果被锁存到输出寄存器中。这种设计将原本可能很长的关键路径切分成了一个独立的流水线阶段，确保了发射阶段在时钟周期开始时就能获得来自寄存器的稳定的已译码指令，有利于提高 cpu 的时钟频率。

（3）**FIFO 管理：**内部通过头指针（head）、尾指针（tail）和计数器（cnt）来管理队列。full 信号用于当缓冲器中剩余空间不足 2 个时，通知取指部件暂停送入新的指令。

2.7 总线模块设计

本模块的设计采用了多状态机并行仲裁的架构，并使用寄存器作为对外设的输出（我在使用组合逻辑作为外设输出时在 120M 就开始容易出现问題），以确保时序的稳定性。

首先通过地址解码确定请求目标是 BaseRam、ExtRam 还是 UART。接着，仲裁逻辑会检查各设备是否空闲，并遵循“数据读取优先、数据写入次之、指令读取最后”的优先级策略进行授权。

为高效管理各设备，模块内部设计了三个独立的状态机。分别为 UART、BaseRam、ExtRam 三种外设进行仲裁。其中，针对来自 Icache 向 BaseRam 的访存请求支持“猝发读取”，能一次性取回整个缓存行。

三、关键技术及特色/亮点

双发射设计：为了避免过于复杂的设计影响频率，所以后端几乎就是简单的将流水线变成两个，数据前递的数量变多，乘法和访存的暂停也是直接同时暂停两条流水线。前端相比五级流水线主要增加了一个指令队列，来解耦前后端，不然每周期发射数量的不稳定会导致前端频繁的暂停导致效率低下。该双发射设计相比我的单发射版本，IPC 提升了约 16%。同时，由于后端设计较为简洁，FPGA 资源占用较少，为实现更高的运行时钟频率创造了有利条件。

四、调试及优化

4.1 测试方法

初步测试主要依靠 2024 年团队赛发布包里的 trace 对比测试框架，但这里的内存和个人赛的 sram 有很大的区别，移植到个人赛发布包时还需要写一个新的总线控制器。

移植到个人赛发布包后也主要依靠 vivado 仿真，官方提供的 tb 可以模拟个人赛的三个外设。仿真时主要看在仿真运行监控程序时串口有没有正常的输出欢迎信息。

4.2 调试和优化过程

主要优化过程就是先在 50MHz 过测试，然后开始加多周期访存（必须要有指令 cache），通过 vivado 时序报告修改关键路径，逐渐开始提频率。频率提到比较高了过后就需要改架构提 ipc，我当时没有优化单发射架构的思路，就选择去重新写一个双发射 cpu，在低频率测试后发现 ipc 相较单发射有提升，就专注优化双发射架构了。改架构时一定要多看测试程序，针对性能测试的三个程序做优化。

双发射在架构上的优化比单发射有更多的可能，可以通过增加更多可以双发射的指令组合，分支跳转指令是否可以和其他指令一起发射，两条流水线是否都是全功能的，在这些上面都可以做优化。

五、复赛编程题设计

测试说明

1. 题目：统计数组中第一个元素出现的次数
给定 32 位无符号整数数组 A，统计数组中 **第一个元素的值** 在数组中出现的次数，并将结果存到结果目标地址，结果为 4 字节无符号整型。
2. 规定：
数组 A 地址为 `0x80400000`，数组长度为 `0x300000` 字节（即 A 的范围是 `0x80400000-0x806FFFFF`）。
结果目标地址为 `0x80700000`。
3. 提示：
测试数据为随机生成，数组中第一个数的值和分布不可预知，必须通过遍历计算得到。

提交说明

1. 实现汇编程序完成指定要求，汇编程序写在仓库根目录下 `asm` 文件夹内的 `user-sample.s` 文件里，最后标记 **最终版本**。
2. `user-sample.s` 文件里提供了一个模板汇编程序（功能是计算斐波那契数并存入 SRAM），选手需要在此基础上改写为本题要求的程序。
3. 允许更改硬件设计，保证独立完成设计即可。如果 `thinpad_top.srscs` 下的文件没有变动，Gitlab 平台不会生成新的 bit，可以节省时间。
4. 比赛分数：未通过测试 0 分，通过测试的按执行时间映射到 **[15,30]**，最短时间为 30 分，最长时间为 15 分。

测试要求

1. 虚拟内存空间为 `0x80000000~0x807FFFFFF`，共 8MB，要求整个内存空间均可读可写可执行。其中：
 - `0x80000000~0x803FFFFFF` 映射到 BaseRAM；
 - `0x80400000~0x807FFFFFF` 映射到 ExtRAM。
2. CPU 字节序为小端序。
3. CPU 时钟使用外部时钟输入，50MHz / 11MHz 两路时钟输入均可使用。
4. 在复位按钮按下时（高电平）CPU 处于复位状态，松开后解除。
5. CPU 复位后从 `0x80000000` 开始取指令执行。

今年的编程题和去年的本质一样就是遍历一遍数组，依靠程序大概能达到 0.022s 左右的成绩，一个一周周期都不浪费连续访存的硬件加速器可以很轻松的达到 0.015s 频率提到 sram 的极限可以达到 0.013s。

算法优化：循环展开效果非常好，不展开的程序只能跑 0.027s，展开四层后可以跑 0.022s。

硬件加速器：使用除法指令 `div` 作为硬件加速器开始的信号，当 `div` 指令进入执行阶段，暂停整个流水线，硬件加速器启动并完全接管 ExtRam 的总线，遍历数组完成计数并写回，然后流水线恢复运行。（抢在监控程序运行前先运行硬件加速器跑出 0s 的成绩在今年已经不被允许了）

六、关键问题及解决方案

问题 1: 用过于复杂的组合逻辑连接外设, 导致在频率提高后运行结果出错。最后改为寄存器直接连接外设解决。

问题 2: 在测试平台测试运行 G 命令时超时, 难以确定具体的问题是什么, 可能是访存不稳定导致, 也有可能是 cpu 核有问题, 最后通过 vivado 仿真模拟串口向 CPU 发送 G 命令, 和学长正确的 CPU 的仿真波形一对比发现是访存暂停时检测到分支预测错误会导致当前分支指令被错误的冲刷掉。这种设计失误应当在早期设计时避免。

七、总结与展望

7.1 竞赛总结

通过本次竞赛, 我对计算机体系结构和超标量技术有了更深入的了解。在提升频率的过程中, 第一次在上板过程中面对了时序收敛问题, 理解了部分 Verilog 代码表达的硬件电路具体是什么样子。在长达一个半月的过测试提性能的过程中, debug 的能力得到了很大的提升。虽然几乎每个 bug 都要花一整天直到凌晨才能发现, 性能的提升也是以天为单位, 但整个比赛的过程还是很有趣的。

7.2 未来展望

今年比赛突然增加了获奖名额, 总共有五个一等奖和十个二等奖, 不知道明年会不会还是这样, 希望明年个人赛可以提供更加丰富全面的测试, 让 dcache 有用武之地, 也希望有更多的人、更多的学校参加到龙芯杯来。

唯一的遗憾就是自己写的乱序 cpu 性能太差了, 经过了比赛后又积累了更多的经验。如果有机会, 明年想参加团队赛实现一个功能完整可以运行 linux 的乱序核。

7.3 备赛经验及建议

1. 进度安排:

这个比赛的官方消息发布的特别慢, 说的五月份给个人赛账号一直到六月份

才给，测试平台一直到七月八号左右才开放，平台开放后一直到 7 月 17 号才发布了发布包，如果等到发布包发布才开始准备的话可能时间已经没有那么宽松了，可能没有充足的大改架构的时间，所以一定要用往年的发布包提前在测试平台开始测试。有问题建议加入比赛官方 QQ 群，有官方和往届参赛选手积极回答问题。

2. 关于超频：

个人赛的频率之所以都非常高是因为比赛没有 wns 限制，所有频率在 200M 以上的 wns 肯定都是负的，但只要能过测试就算是有效成绩。超频对于用的 FPGA 资源比较少的 cpu 非常有利，因为用的资源少，布线更容易，就算是 wns 为负过测试的概率也很高。但对于非常复杂的乱序 cpu 或者用寄存器写的容量很大的 cache 的 cpu，频率是很难提高的。所以现在的个人赛最有性价比的还是单发射，但要追求更高的性能还是要双发射，今年 LA 的三个一等奖都是双发射并且频率在 230M 以上。明年可能会增加 wns 限制，增加了 wns 限制可能可以让更复杂的 cpu 有更大的优势。

3. 关于硬件加速器：

比赛前一定要提前准备好硬件加速器的接口，不然在赛场上从零开始写很有可能会遇到 bug，在线生成 bit 流一次要花 40 分钟，如果现场改硬件出现 bug 很难在规定时间内找到 bug 并重新在线编译。

今年的题目和去年的太像了，如果提前准备了去年的硬件加速器今年在比赛时可以很快的写出来，不过明年可能不会再像今年那么简单了。

4. 决赛的准备：

决赛时在线测试平台使用的人太多了，在线编译会变得特别慢，而且还要排队，所以一定要提前在 gitlab 平台上准备好自己各个版本各个频率的分支，并且都在线编译完成，这样决赛时改汇编，在线编译一次只需要 9 分钟，等测试通过后再考虑改核压力不会那么大，改核一次就要编译 40 分钟，如果一直过不了测试压力会非常大。

5. 避坑：

如果个人赛不改测试的话，dcache 还是不需要，既影响频率命中率也不高，而且个人赛的测试平台会直接检测内存中的数据对不对，dcache 用写回策略的话如何将更改的数据写回主存也是一个问题。